

Software Engineering - Lab 4:

Applying Software Design Patterns

Objective: The purpose of this assignment is to gain practical experience implementing a variety of common software design patterns. You will construct a small command-line e-commerce application, where each feature will be built using a specific design pattern to solve a particular problem. This assignment focuses purely on code architecture and design.

Context: Design patterns are reusable, well-documented solutions to commonly occurring problems within a given context in software design. They are not finished designs that can be transformed directly into code. They are descriptions or templates for how to solve a problem that can be used in many different situations. We will explore Creational, Structural, and Behavioral patterns.

Prerequisites

- A solid understanding of JavaScript classes and modules.
- Node.js installed to run the application.

Submission

The **URL** to your public GitHub repository.

`https://github.com/<username>/git-e-commerce-patterns-<StudentID>-<FullName>`

One ZIP named: `submission_lab4_<StudentID>_<YourName>.zip` containing:

- Either **screenshots** or a **run_log.txt** showing the expected console output for each part (2A, 2B, 3A, 3B, 4A, 4B, 4C). Suggested file names: `part2a_singleton.png`, `part3b_facade.png`, etc., or a single `run_log.txt` captured from node `index.js`.

Part 1: Project Setup

1. **Use a New Repository or Branch:** Start with a clean slate. Create a new GitHub repository or a new branch for this assignment.
`git-e-commerce-patterns-<StudentID>-<FullName>`
2. **Add Project Files:** Download and add the e-commerce-patterns project files to your local repository.
3. **Install Dependencies:** There are no external libraries, but to ensure your project is set up, run:
`npm install`
4. **Examine the Structure:** The patterns directory is organized by the three main categories of design patterns. You will be implementing the code within these files. The `index.js` file is where you will run your simulation.

Part 2: Creational Patterns - How Objects are Created

Creational patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.

A. The Singleton Pattern

Goal: Ensure that a class has only one instance and provide a global point of access to it. Our shopping cart is a perfect example; we only want one cart instance for the entire application.

1. **Implement CartService.js:** Open patterns/creational/CartService.js. Follow the // TODO comments to implement the Singleton pattern. You need to ensure that no matter how many times the CartService is imported or instantiated, it always returns the *same* instance.
2. **Test in index.js:** In index.js, uncomment the "Part 2A" section. Run node index.js. The console should show true, proving that cart1 and cart2 are the same instance.

B. The Factory Pattern

Goal: Define an interface for creating an object, but let subclasses decide which class to instantiate. This is useful when a method cannot anticipate the class of objects it must create.

1. **Implement ProductFactory.js:** Open patterns/creational/ProductFactory.js. Complete the createProduct method. Based on the type string passed to it, it should return an instance of a Book, Electronic, or a generic Product.
2. **Test in index.js:** Uncomment the "Part 2B" section and run the file. You should see the details of a book and an electronic item logged to the console, created by your factory.

Part 3: Structural Patterns - How Objects Relate to Each Other

Structural patterns explain how to assemble objects and classes into larger structures, while keeping these structures flexible and efficient.

A. The Decorator Pattern

Goal: Attach new behaviors to objects by placing these objects inside special wrapper objects that contain the behaviors. This allows behavior to be added to an individual object without affecting the behavior of other objects from the same class.

1. **Implement ProductDecorator.js:** Open patterns/structural/ProductDecorator.js. Complete the GiftWrapDecorator and ExtendedWarrantyDecorator. Each decorator should take a product object, add a description of the new feature, and increase its price.
2. **Test in index.js:** Uncomment the "Part 3A" section. This code will create a book, wrap it with gift wrap, then wrap it again with an extended warranty. When you run the file, you should see the final price correctly calculated and the description updated.

B. The Facade Pattern

Goal: Provide a simplified interface to a library, a framework, or any other complex set of classes. A Facade reduces the complexity of working with a subsystem.

1. **Implement CheckoutFacade.js:** Open patterns/structural/CheckoutFacade.js. Implement the placeOrder method. This method should orchestrate calls to the (mock) InventoryService, PaymentService, and ShippingService to perform all the steps of a checkout with a single method call.
2. **Test in index.js:** Uncomment the "Part 3B" section and run the file. You should see a sequence of logs from the different services, all initiated by the single call to your facade.

Part 4: Behavioral Patterns - How Objects Communicate

Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects.

A. The Strategy Pattern

Goal: Let you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable. We will use this to handle different shipping calculation methods.

1. **Implement ShippingStrategy.js:** Open patterns/behavioral/ShippingStrategy.js. Complete the FlatRateStrategy and WeightBasedStrategy classes. Each should have a calculate method that returns a shipping cost based on the provided package details. Then, complete the ShippingCalculator class so it can be configured with a strategy.
2. **Test in index.js:** Uncomment the "Part 4A" section. This code will calculate the shipping cost for the same package using two different strategies. Verify the output is correct.

B. The Command Pattern

Goal: Turn a request into a stand-alone object that contains all information about the request. This transformation lets you pass requests as a method argument, delay or queue a request's execution, and support undoable operations.

1. **Implement Command.js:** Open patterns/behavioral/Command.js. Implement the AddToCartCommand class. It should have an execute method that adds a product to the cart service and an undo method that removes it.
2. **Test in index.js:** Uncomment the "Part 4B" section. You will see a product added, the action undone, and then re-executed.

C. The Observer Pattern

Goal: Define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.

1. **Implement Observer.js:** Open patterns/behavioral/Observer.js.
 - o Complete the OrderTracker (the Subject). It needs methods to addObserver, removeObserver, and notifyObservers.
 - o Complete the EmailNotifier and DashboardNotifier (the Observers). They each need an update method that will be called by the Subject.
2. **Test in index.js:** Uncomment the "Part 4C" section. When the order status is updated in the tracker, both the email and dashboard notifiers should automatically log a message to the console.

Grading Rubric

Part	Criteria	Points
1. Project Setup & Repo	Public GitHub repository created and named exactly: git-ecommerce-patterns-<StudentID>-<FullName>.	10
Subtotal		10
2. Creational Patterns	2A. Singleton (CartService): one instance enforced; demonstrates correct behavior in index.js output.	10
	2B. Factory (ProductFactory): returns correct concrete types; demonstrates via index.js output.	10
Subtotal		20
3. Structural Patterns	3A. Decorator (GiftWrap & ExtendedWarranty): price and description updated cumulatively; shown in output.	12
	3B. Facade (CheckoutFacade): orchestrates inventory, payment, shipping; single call produces expected sequence of logs.	13
Subtotal		25
4. Behavioral Patterns	4A. Strategy (ShippingStrategy & ShippingCalculator): interchangeable strategies produce correct costs.	12
	4B. Command (AddToCartCommand): execute/undo work correctly with cart state changes.	10
	4C. Observer (OrderTracker, EmailNotifier, DashboardNotifier): observers notified on status change.	13
Subtotal		35
5. Demonstration & Packaging	Evidence of successful runs (screenshots or run_log.txt) showing Parts 2A - 4C outputs.	5
	Submission packaging follows naming conventions; repository is tidy (sensible commits, no build artifacts).	5
Subtotal		10
TOTAL		100

Good luck!